PROJECT REPORT

Privacy-Preserving Medical Record Sharing System

MedRec+: Encrypted, Pseudonymous and Time-Bound Access Control for Blockchain-Based Health Records

Submitted by

#	Name	Roll Number
1	A.Palvash Kumar	S20230010029
2	A.Naga Venkatesh	S20230010001
3	M.Jashwanth	S20230010145

Base paper

A. Azaria, A. Ekblaw, T. Vieira and A. Lippman, "MedRec: Using Blockchain for Medical Data Access and Permission Management," *2nd Int. Conf. on Open and Big Data (OBD)*, 2016, pp. 25–30.

Source of improvement ideas

M. Li, S. Yu, Y. Zheng, K. Ren and W. Lou, "Scalable and Secure Sharing of Personal Health Records in Cloud Computing Using Attribute-Based Encryption," *IEEE TPDS*, 24(1), 2013.

October 2026

Abstract: MedRec keeps medical records in each hospital's own database and uses Ethereum smart contracts to hold pointers, integrity hashes and access permissions. We re-implemented MedRec on a local Hardhat chain with a Node.js gatekeeper and an SQLite provider database, and showed four weaknesses with scripted demonstrations: the off-chain records are stored in plaintext, on-chain metadata links patients to providers, access grants never expire, and there is no emergency access route. We then built **MedRec+**, which adds envelope encryption (AES-256-GCM with X25519 key wrapping), pairwise pseudonyms with salted selector commitments, time-bound grants with key rotation, and an audited break-glass role. In our local evaluation, clinical plaintext exposure in the database fell from 100% to 0%, on-chain identity attribution fell from 100% to 0% in the observer model we tested, and the extra cost was +3.0% gas to share a record and under 75 ms of cryptography for a 50 MB record. All 42 automated tests pass.

1. Introduction

A patient's medical history is usually spread across several hospitals, clinics and laboratories, each with its own database. Sharing records between them is slow, and the patient rarely knows who has seen their data. Central record repositories solve the fragmentation problem but create a single high-value target for attackers and require everyone to trust one operator. The cyber security problem we address is therefore: *how can records be shared between providers under the patient's control, without exposing them to storage insiders, to observers of the sharing system, or to people whose access should have ended?*

Putting records directly on a blockchain does not solve this. A public ledger is readable by every node, it cannot erase data (which conflicts with privacy regulation), and storing large files on-chain is very expensive. The design space is therefore "data off-chain, permissions on-chain", which is exactly what MedRec proposes. Our objectives were to (i) re-implement MedRec faithfully, (ii) find and demonstrate its limitations, (iii) design and implement improvements, and (iv) measure their benefit and cost against the baseline.

1.1 Why we selected this paper

We shortlisted two well-cited papers on the topic and scored them on six criteria (1 = poor, 5 = strong). Table 1 summarises the comparison.

Table 1: Paper selection: MedRec versus Li et al. (attribute-based encryption for PHRs).

Criterion	MedRec (2016)	Li et al. (2013)
Implementable by a student team in 4–5 weeks	5	2
Mainstream tooling that installs cleanly	5	2
Produces measurable results (gas, latency, throughput)	4	4
Limitations are clearly stated and citable	5	4
Room for a meaningful, implementable improvement	5	4
Fit with a JavaScript stack (Hardhat, Node.js)	5	2
Total (out of 30)	**29**	**18**

We chose **MedRec** for three reasons. First, it is buildable: three smart contracts, a gatekeeper service and a database can be implemented, tested and benchmarked in a few weeks, whereas a faithful reproduction of Li et al. needs pairing-based cryptography, multi-authority ABE and proxy re-encryption. Second, the authors themselves point to the gaps: contract-level encryption was not added to the prototype, no encryption of the off-chain data is described, and they acknowledge that transaction patterns remain open to forensic analysis. That gave us a well-defined improvement target. Third, MedRec reports no performance evaluation (it lists simulation of system efficiency as future work), so our measurements add something new. We used Li et al. not as a system to rebuild but as the source of three ideas: encrypting each file under its own random key, break-glass emergency access, and deferring re-encryption on revocation.

2. Overview of the Existing Technique (MedRec)

MedRec separates *where data lives* from *who may see it*. Records stay in the provider's own database behind a *gatekeeper* server. The blockchain stores only small items: a pointer to the record, a hash that proves it has not been changed, and the list of addresses allowed to view it. Three contracts implement this (Table 2).

Table 2: MedRec smart contracts.

Contract	Role
Registrar (RC)	Maps a participant's identity string (name or ID) to an Ethereum address and to their Summary contract.
Summary (SC)	Per-participant index of every patient–provider relationship, with a status flag used for notifications.
Patient–Provider Relationship (PPR)	One per relationship. Holds data pointers (query selector + hash), the gatekeeper location and viewer permissions.

Workflow: (1) A provider adds a record to its database and calls `addPointer` on the PPR with the record's selector and hash. (2) The patient's Summary contract is flagged, so the patient sees the new record. (3) The patient calls `share` to permit a doctor or lab. (4) The viewer sends a signed request to the gatekeeper, which checks `canView` on-chain, runs the query and returns the record; the viewer checks the hash against the on-chain value. The approach gives the patient a single view of all relationships, an immutable permission history and no central data store. Figure 1 shows the architecture, with a summary of the MedRec+ changes.

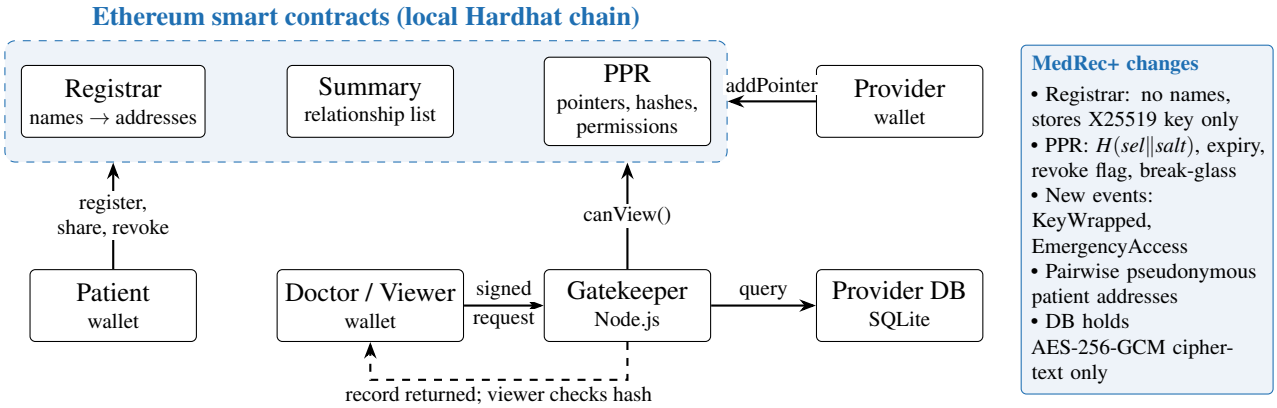


Figure 1: MedRec architecture as we implemented it. The box on the right lists what MedRec+ changes.

3. Implementation of the Existing Approach

Stack: Contracts are written in Solidity 0.8.24 and run on a local Hardhat 2 chain; tests use Mocha through `hardhat-toolbox`; clients and the gatekeeper use Node.js with `ethers` v6, Express 5 and `better-sqlite3`. The provider database holds six synthetic patient records (name, date of birth, blood type, diagnosis, medication, visit date); no real patient data was used.

Components built: The three contracts (`Registrar.sol`, `Summary.sol`, `PPR.sol`) follow the paper. A command-line client (`client/cli.js`) implements the add-record, retrieve and share flows for patient, provider and viewer, and a gatekeeper service (`gatekeeper/index.js`) verifies each request. The gatekeeper recovers the signer from a signed {PPR, selector, nonce, timestamp} message, rejects reused nonces and stale timestamps (replay protection), calls `canView` on-chain, runs a parameterised query, and compares the SHA-256 of the result with the on-chain hash before returning it.

Deviations, stated honestly: (a) MedRec stores raw query strings on-chain and the gatekeeper executes them; running query text taken from a public transaction invites SQL injection, so we used selectors mapped to parameterised queries. (b) Proof-of-work mining and the researcher "bounty" model were replaced by a local chain with emulated block times (instant, 2 s and 12 s). (c) Identity strings were kept on-chain as in the paper, because that is one of the behaviours we later critique.

Baseline results: 21 automated tests pass (15 contract tests and 6 gatekeeper integration tests covering authorisation, replay and tampering). Over 30 runs, registration cost 336,401 gas on average and a record pointer used 132 bytes of calldata. Transaction latency was 0.67 ms with instant mining but 2,014.6 ms and 12,011.9 ms at 2 s and 12 s block intervals, so confirmation time is set by the consensus interval rather than by contract execution. The gatekeeper served up to 509 requests/s with 100 concurrent clients, with no errors.

4. Challenges and Limitations

We scripted four weakness demonstrations (D1–D4) against our baseline. Together with the authors' own stated limitations they give the challenges in Table 3.

Table 3: Challenges found in MedRec, with evidence and impact.

Challenge	Evidence	Security impact
C1 Plaintext off-chain data	D1: opening the SQLite file shows 6/6 records in clear text. D4: calling the data layer directly bypasses the contract check.	A database dump, an insider or a compromised gatekeeper exposes every record. Protection depends on server code, not cryptography.
C2 Linkable on-chain metadata	D2: an observer script reads Registrar names, PPR links, viewer lists and selectors such as `records:oncology`.	Anyone running a node learns who is treated where, who was given access and hints about the condition (100% attribution).
C3a No expiry on access	Grants last until manually revoked; the paper only suggests expiry. D3: a revoked viewer keeps data already fetched.	A previous doctor keeps standing access after treatment ends.
C3b No emergency route	No mechanism for access without a prior grant.	An unconscious patient's allergies or blood group cannot be released in time.
C4 Scalability	Listed as unresolved by the authors; latency follows block time (12 s per confirmation).	Measured and discussed only; not addressed by our design.

5. Proposed Approach: MedRec+

Threat model: We defend against four adversaries: A1, a curious storage operator who can read the database but holds no keys; A2, a passive chain observer who reads all state, calldata and events; A3, a revoked or expired viewer who tries to fetch again; and A4, an impostor claiming an emergency. The provider that creates a record is a trusted custodian, and endpoint malware and network traffic analysis are out of scope. Our design goal was to fix C1–C3 with standard, fast cryptography rather than heavy ABE.

E1 – Envelope encryption (fixes C1): Each record is encrypted with a fresh 256-bit data encryption key (DEK) using AES-256-GCM, with the record ID bound as authenticated associated data (AAD) so a ciphertext cannot be swapped onto another record. The DEK is then *wrapped* separately for each authorised reader: an ephemeral X25519 key agreement with the reader's public key, HKDF-SHA256 to derive a wrapping key, and AES-GCM to encrypt the DEK. Only ciphertext reaches the database; the on-chain hash is the SHA-256 of the ciphertext. Wrapped keys are published as `KeyWrapped` contract events rather than in storage, which we show is much cheaper.

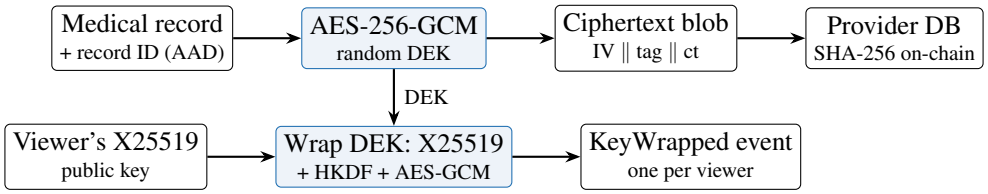


Figure 2: E1 envelope encryption: bulk data under a per-record key, the key wrapped per reader.

E2 – Reduced linkability (fixes C2): `RegistrarPlus` stores no identity strings. A patient uses a separate pseudonymous address for each provider relationship, so relationships cannot be joined by address. Selectors are stored as salted commitments $c = \mathrm{SHA256}(\mathit{selector} \,\|\, \mathit{salt})$, with the salt shared only with the counterparty, so an observer sees an opaque 32-byte value instead of "records:cardiology".

E3 – Time-bound grants and key rotation (fixes C3a): A grant is now {viewer, selector commitment, `expiresAt`, `revoked`}, and `canView` returns true only if `block.timestamp < expiresAt` and the grant is not revoked. When a record is updated after a revocation, a new DEK is generated and wrapped only for the remaining viewers, so a revoked viewer cannot decrypt future versions even if they obtain the ciphertext.

E4 – Audited break-glass (fixes C3b): The patient designates an emergency role (e.g. an emergency department) and flags a minimal emergency record set (allergies, blood group, medication). Only that role can call `emergencyAccess`, which requires a duration between 1 second and 24 hours, sets an automatic expiry and emits a public `EmergencyAccess` event. Every use is therefore auditable, and the patient can revoke it afterwards.

6. Implementation of the Proposed Approach

MedRec+ was built alongside the baseline so that the same benchmark harness could run both modes. Table 4 lists what changed.

Table 4: Implementation of the MedRec+ enhancements.

File	What it implements
`crypto-envelope.js`	E1: `encryptRecord`, `wrapKey`, `unwrapKey`, `decryptRecord` and `computeSelectorCommit`, built only on Node's `crypto` module (no third-party crypto). Ciphertext adds a constant 28 bytes (12-byte IV + 16-byte tag).
`RegistrarPlus.sol`	E2: registration by address and X25519 public key only; no identity strings.
`PPRPlus.sol`	E1–E4: pointers keyed by selector commitments with timestamps; `Grant` struct with expiry and revoked flag; `share` (event) and `shareWithStorage` (for comparison); O(1) `revoke`; emergency role, emergency selector set and `emergencyAccess` with a 24-hour cap.
`gatekeeper/index.js`	Dual mode: in MedRec+ mode it checks the grant on-chain, returns only ciphertext from the `records_encrypted` table and resolves the caller's wrapped key from events.
`deploy-plus.js`	Seeds an encrypted database with no clinical plaintext columns and deploys the MedRec+ contracts.
`medrec-plus.test.js`	21 tests: registration, pointers, grants, expiry, revocation, break-glass, gatekeeper queries and DEK rotation.
Analysis and bench scripts	`observer-compare.js` (H2) and `security-experiments.js` (H1, H3–H7), writing CSV/JSON to `bench/results/`.

The core permission check in `PPRPlus.sol` shows how E3 and E4 combine:

```
function canView(address who, bytes32 selectorCommit) external view returns (bool) {
    if (who == patient || who == provider) return true;
    if (who == emergencyRole && emergencyRole != address(0) &&
        block.timestamp < emergencyExpiresAt && emergencySelectors[selectorCommit])
        return true;                                    // E4: audited break-glass
    Grant memory g = grants[who][selectorCommit];
    if (!g.exists || g.revoked) return false;           // E3: explicit revocation
    return block.timestamp < g.expiresAt;               // E3: automatic expiry
}
```

The full regression suite now has **42 tests** (15 baseline contract, 6 gatekeeper integration and 21 MedRec+), all passing in about 620 ms via `npm test`.

7. Results and Analysis

We tested seven hypotheses (H1–H7) with the same harness for both systems. Table 5 gives the security results and Figures 3–5 the costs. Gas values are transaction receipts under identical chain state.

Table 5: Security results: baseline MedRec versus MedRec+.

Hypothesis / metric	Baseline MedRec	MedRec+
H1 Clinical plaintext in DB	100% readable (6/6)	**0% readable (0/6)**
H2 On-chain identity attribution	100%	**0% (0/3)** in our observer model
H2 Plaintext selectors leaked	2	**0** (salted commitments)
H2 Largest address cluster	2 relationships	**1** (pairwise pseudonyms)
H5 Access after expiry / revoke	No expiry exists	**100% blocked**, 0 leaks
H6 Break-glass auditability	Not supported	**100/100** events logged
H7 Tamper: flipped ciphertext byte	SHA-256 mismatch only	**Rejected** by GCM and hash
H7 Tamper: record swapped to other ID	Not detected	**Rejected** by AAD

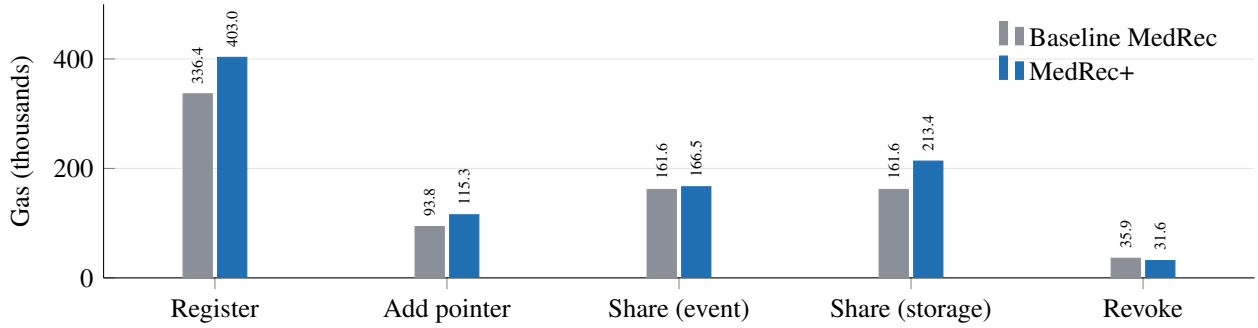


Figure 3: Gas per operation (H4). Creating a PPR, not shown, rose from 1,160,475 to 1,268,715 gas (+9.3%).

Gas (H4): Registration costs 19.8% more because the X25519 public key is stored, and adding a pointer 22.9% more for the commitment and timestamp. Sharing is the most frequent operation, and publishing the wrapped key as an event keeps its overhead to **+3.0%** (166,498 gas); writing it to storage would cost +32.0%. Storage is therefore 46,862 gas (28.1%) more expensive than events. Revocation became **12.0% cheaper** because it sets a single flag instead of shifting an array.

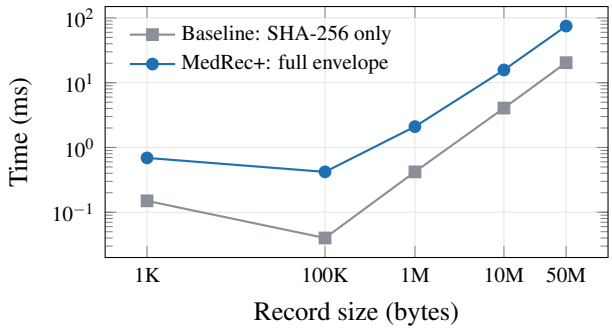


Figure 4: Cryptographic time vs. record size (H3).

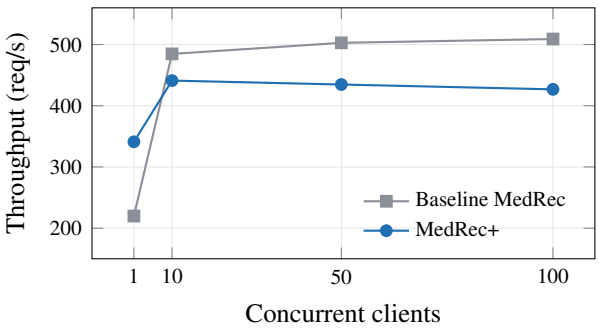


Figure 5: Gatekeeper throughput (B4).

Latency and throughput (H3, B4): The full envelope (encrypt, wrap, unwrap, decrypt) took 0.69 ms for 1 KB, 2.10 ms for 1 MB and 74.97 ms for 50 MB, with a fixed 28-byte ciphertext expansion. The baseline line measures only the SHA-256 hash, so the gap is a comparison of two different workloads rather than a pure overhead; sub-millisecond baseline points are within timer noise. Either way, the cost is tiny next to a 12 s block confirmation. Under 100 concurrent clients the MedRec+ gatekeeper served 426.8 req/s against 509.0 for the baseline (p95 latency 304.9 ms versus 229.9 ms), a 16% reduction caused by key resolution and commitment checks; with one client MedRec+ was faster because it looks records up by primary key. No request failed in either mode.

Table 6: Feature summary: what the improvement changes.

Feature	Baseline MedRec	MedRec+
Off-chain record storage	Plaintext in SQLite	AES-256-GCM ciphertext, per-record DEK
Identity on chain	Names in Registrar	Pairwise pseudonymous addresses
Query selectors on chain	Readable text	Salted SHA-256 commitments
Access expiry	None	On-chain `expiresAt`, checked every request
After revocation	Old access path only	DEK rotation for new versions
Emergency access	Not supported	Role-restricted, max 24 h, public audit event
Tamper detection	SHA-256 hash	SHA-256 + GCM tag + record-ID binding
Automated tests	21	42 (all passing)

Discussion: MedRec+ removes the two largest privacy leaks of MedRec at a moderate, predictable cost. Some exposure remains, and we report it plainly: the database still shows patient IDs, timestamps and ciphertext lengths; pseudonymous addresses can be linked if they are funded from one wallet or timed with a visit; `Shared` events still show that two addresses interacted; revocation cannot recall data already decrypted; and the 0% attribution figure holds for our simulated observer, not for every possible re-identification attack.

8. Conclusion

We re-implemented MedRec, demonstrated that its off-chain records are plaintext, its on-chain metadata is linkable, and its grants neither expire nor allow emergency access, and then built MedRec+ to address these gaps with standard primitives. In our local evaluation MedRec+ cut database plaintext exposure from 100% to 0% and on-chain identity attribution from 100% to 0%, enforced expiry and revocation with zero leaks, logged every emergency access, and detected all tampering, while adding only 3.0% gas to sharing and under 75 ms of cryptography for a 50 MB record. Future work includes patient key recovery (social recovery or provider escrow), proxy re-encryption so that sharing does not require the patient to unwrap the key, attribute-based policies, zero-knowledge access proofs to hide who shares with whom, and evaluation on a public testnet.

References

[1] A. Azaria, A. Ekblaw, T. Vieira and A. Lippman, "MedRec: Using Blockchain for Medical Data Access and Permission Management," in *Proc. 2nd Int. Conf. on Open and Big Data (OBD)*, 2016, pp. 25–30, doi:10.1109/OBD.2016.11.

[2] M. Li, S. Yu, Y. Zheng, K. Ren and W. Lou, "Scalable and Secure Sharing of Personal Health Records in Cloud Computing Using Attribute-Based Encryption," *IEEE Trans. Parallel and Distributed Systems*, vol. 24, no. 1, pp. 131–143, 2013, doi:10.1109/TPDS.2012.97.

[3] J. Benaloh, M. Chase, E. Horvitz and K. Lauter, "Patient Controlled Encryption: Ensuring Privacy of Electronic Medical Records," in *Proc. ACM Cloud Computing Security Workshop (CCSW)*, 2009.

[4] J. Liu, M. Wu, R. Sun, X. Du and M. Guizani, "BMDS: A Blockchain-based Medical Data Sharing Scheme with Attribute-Based Searchable Encryption," in *Proc. IEEE ICC*, 2021, pp. 1–6, doi:10.1109/ICC42927.2021.9500966.

[5] Hardhat, ethers.js v6 and Node.js `crypto` module documentation (AES-GCM, X25519, HKDF).